

IRANet

Spanish version: [README.es.md](#)

IRANet is a read-only observability and system introspection platform for Linux servers. It discovers what is really running on each server and exposes that information through a backend API and a web frontend.

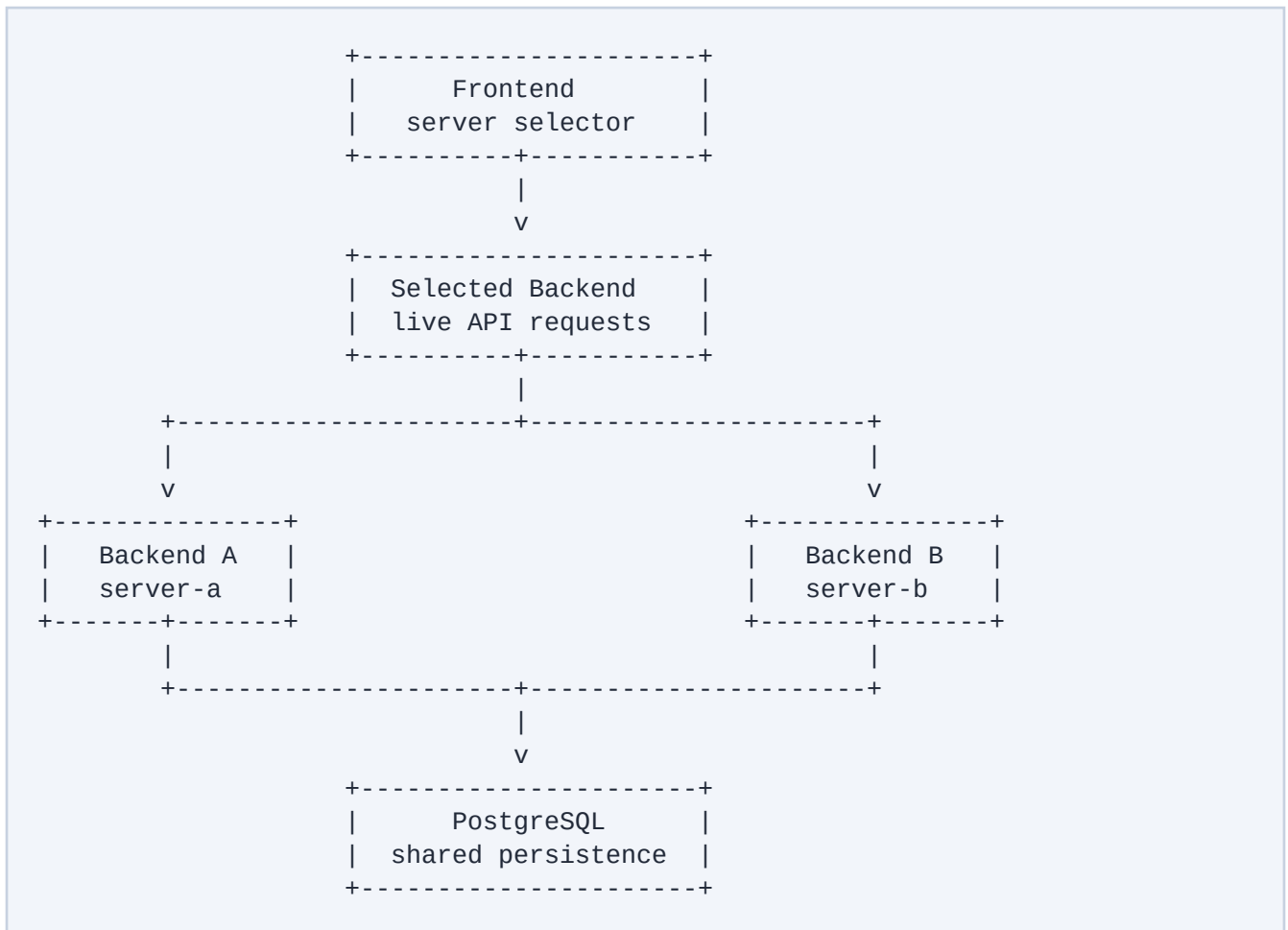
The platform is intentionally read-only: it gives visibility into servers, processes, services, packages, logs, and metrics without allowing remote execution from the IRANet UI itself.

Overview

IRANet is designed for a deployment model with:

- one shared PostgreSQL database
- one frontend
- one IRANet backend per monitored server

Each backend monitors only its own host and writes data into the same PostgreSQL database. The frontend can then switch between servers and talk to the selected backend for live data.



What IRANet Collects

IRANet can discover and expose:

- Docker containers and services
- systemd services
- detected databases
- running processes
- long-lived applications
- system users
- installed packages and package history
- system logs and application logs
- system metrics
- per-application runtime metrics
- alert history

Final Architecture

IRANet now follows a single architecture model.

Backend

Each backend:

- runs on exactly one server
- monitors only that local server
- writes all persistent data to the shared PostgreSQL database
- registers itself in the servers table
- updates heartbeat metadata periodically
- serves live requests for its own host only

Frontend

The frontend:

- boots from any reachable backend URL
- loads the server list from PostgreSQL
- lets the user choose which server to inspect
- uses the selected backend URL for live views

PostgreSQL

PostgreSQL is mandatory.

All backends must point to the same IRA_DATABASE_DSN.

That shared database stores:

- servers
- applications
- application metrics
- system metrics
- alerts
- extensions

Important Runtime Rules

- IRA_DATABASE_DSN is required
- IRA_SERVER_ID is required
- each backend must have a unique IRA_SERVER_ID
- each backend must point to the same PostgreSQL database
- live endpoints only operate on the local backend's server
- historical data is shared through PostgreSQL

Requirements

Backend

- Python 3.11+
- PostgreSQL 16+
- Linux host

Frontend

- Node.js 18+
- npm, pnpm, or yarn

Recommended deployment

- Docker Engine
- Docker Compose v2

Docker Deployment

The repository provides three Compose stacks:

- docker/compose.db.yml
- docker/compose.backend.yml
- docker/compose.frontend.yml1. Central PostgreSQL host

2. One backend per monitored server

```
cd docker
cp .env.backend.example .env.backend
docker compose --env-file .env.backend -f compose.backend.yml up -d --build
```

3. Single frontend host

```
cd docker
cp .env.frontend.example .env.frontend
docker compose --env-file .env.frontend -f compose.frontend.yml up -d --build
```

Notes

- compose.backend.yml is reused on every monitored server
- every backend uses the same PostgreSQL DSN
- the frontend only needs one backend URL for bootstrap
- after bootstrap, the frontend can switch to the selected server backend URL

Local all-in-one setup

If you want to run everything on a single machine for development:

```
cd docker
docker compose \
  -f compose.db.yml \
  -f compose.backend.yml \
  -f compose.frontend.yml \
  up -d --build
```

Or use the helper scripts:

```
cd docker
./up.sh
./down.sh
```

Backend Environment Variables

Required

| Variable | Description | |---|---| | IRA_DATABASE_DSN | Shared PostgreSQL DSN, example postgresql+asyncpg://user:pass@db:5432/iranet | | IRA_SERVER_ID | Unique identifier for this backend/server |

Optional

| Variable | Default | Description | |---|---|---| | IRA_SERVER_NAME | hostname
| Human-readable server name | | IRA_AGENT_BASE_URL | empty | Public backend
URL for this server | | IRA_AGENT_PORT | 8000 | Backend listen port | |
IRA_SERVER_ENVIRONMENT | production | Environment label | |
IRA_SERVER_CAPABILITIES | built-in list | Comma-separated capability list | |
IRA_CONFIG_PATH | app/config/ira.config.json | JSON config path |

Frontend Environment Variables

| Variable | Default | Description | |---|---|---| | VITE_API_BASE_URL |
http://localhost:8000 | Bootstrap backend URL | | VITE_SERVER_ID | empty |
Initially selected server |

Running The Backend Locally

```
cd ira
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

export IRA_DATABASE_DSN="postgresql+asyncpg://user:password@localhost:5432/iranet"
export IRA_SERVER_ID="server-01"
export IRA_SERVER_NAME="Server 01"
export IRA_AGENT_BASE_URL="http://127.0.0.1:8000"

python3 -m app.main
```

Running The Frontend Locally

```
cd frontend
npm install
export VITE_API_BASE_URL="http://localhost:8000"
npm run dev
```

Installing A Backend Remotely From Your Own Panel

This is the most important operational flow if you already have your own frontend or admin panel that manages servers.

The intended model is:

- your panel stores server credentials and metadata
- your backend calls IRANet to generate an install command
- your backend executes that command over SSH on the target server
- the new backend starts, connects to PostgreSQL, and registers itself automatically

Step 1. Register or update the server record

You can pre-register a server in IRANet using the servers API.

Example body:

```
{
  "id": "server-01",
  "hostname": "server-01",
  "display_name": "Production 01",
  "agent_base_url": "http://10.0.0.21:8000",
  "environment": "production",
  "capabilities": [
    "system",
    "processes",
    "services",
    "logs",
    "packages",
    "users",
    "metrics"
  ]
}
```

Step 2. Ask IRANet for a Docker install command

Use:

```
GET /servers/{server_id}/install-command
```

Supported query params:

- database_dsn required
- image optional
- server_name optional
- backend_base_url optional
- backend_port optional
- environment optional
- capabilities optional
- repo_url optional, only used to locate install.sh
- branch optional, only used to locate install.sh

Example:

```
curl "http://iranet-api:8000/servers/server-01/install-command?database_dsn=postgresql%2B"
```

The API returns a Docker-based install command that your own backend can execute over SSH.

Step 3. Execute the command over SSH

Your frontend should not execute SSH directly. Your own backend should do that.

Recommended flow:

- user clicks Install IRANet
- your frontend calls your backend
- your backend requests /servers/{server_id}/install-command
- your backend executes response.command over SSH on the target server
- your backend polls /servers/{server_id} until heartbeat is visible

Pseudo-flow:

```
const installResp = await fetch(`${IRANET_API}/servers/${serverId}/install-command?...`);
const installData = await installResp.json();

await ssh.execCommand(installData.command);
```

Quick Start In English

If you already have your own server management panel, this is the shortest installation flow for a remote server:

- Create or update the server record in IRANet.
- Request a Docker install command from IRANet.
- Execute that command over SSH on the target server.
- Wait for the backend heartbeat.

Example:

```
curl "http://iranet-api:8000/servers/server-01/install-command?database_dsn=postgresql%2B"
```

IRANet returns a command like this:

```
curl -sL https://github.com/markush0f/IRANet/raw/main/ira/install.sh | bash -s -- --serve
```

Then execute it on the remote server through SSH from your own backend.

Check status on the target server:

```
sudo systemctl status iranet-backend  
sudo journalctl -u iranet-backend -f
```

What the installer does

The generated command downloads ira/install.sh, then:

- pulls the Docker image
- creates or replaces the iranet-backend systemd service
- sets all required IRANet environment variables
- starts the backend
- lets it register itself through heartbeat

Docker image used

Default image:

```
ghcr.io/markush0f/iranet/ira-backend:latest
```

Result on the target server

The remote machine ends up with a systemd service named:

```
iranet-backend
```

Useful commands on the target server:

```
sudo systemctl status iranet-backend  
sudo journalctl -u iranet-backend -f
```

Live vs Historical Data

This distinction is important.

Live data

Live endpoints must hit the selected backend directly.

Examples:

- system snapshot
- running processes
- users
- package state
- application discovery
- log streaming
- runtime process inspection

Historical data

Historical data is shared through PostgreSQL.

Examples:

- stored metrics
- alert history
- registered applications
- server inventory
- application metrics history

Security Notes

- IRANet itself is read-only
- the IRANet UI does not execute commands on servers
- if you install remotely from your own panel, SSH execution belongs to your own backend, not to IRANet frontend
- deploy behind a reverse proxy or inside trusted networks
- secure PostgreSQL access properly

Current Terminology Note

Some internal names still use `agent_base_url` for historical reasons.

In the current architecture, that field should be understood as:

- the public backend URL of that server

License

MIT